Markov Model

1. Import Library

```
from collections import defaultdict
```

Purpose

defaultdict automatically creates dictionary entries when a new key is encountered.

Example:

```
transitions["love"]["machine"] += 1
```

No manual initialization is required.

2. Training Corpus

```
corpus = [
    "I love machine learning",
    "I love deep learning",
    "I study machine learning",
    "Machine learning is interesting"
]
```

Purpose

The corpus is the dataset used to train the Markov Model.

Training sentences:

1. I love machine learning
2. I love deep learning
3. I study machine learning
4. Machine learning is interesting

The model learns word-to-word transitions from these sentences.

3. Create Transition Table

```
transitions = defaultdict(lambda: defaultdict(int))
```

Structure

```
{
    current_word : {
```

```
        next_word : frequency
    }
}
```

Example:

```
{
    "i": {
        "love": 2,
        "study": 1
    }
}
```

This table stores how often one word follows another.

4. Build the Markov Model

```
for sentence in corpus:
    words = sentence.lower().split()
```

What Happens?

Each sentence is:

1. Converted to lowercase
2. Split into individual words

Example:

```
"I love machine learning"
```

becomes

```
['i', 'love', 'machine', 'learning']
```

5. Count State Transitions

```
for i in range(len(words) - 1):
```

The loop examines every pair of consecutive words.

```
current_state = words[i]
next_state = words[i + 1]
```

Example:

```
i → love
love → machine
machine → learning
```

Frequency is updated using:

```
transitions[current_state][next_state] += 1
```

6. Transition Table After Training

The learned transitions become approximately:

```
{
 'i': {'love': 2, 'study': 1},

 'love': {
     'machine': 1,
     'deep': 1
 },

 'study': {'machine': 1},

  'machine': {'learning': 3},

  'learning': {
      'is': 1
  },

  'deep': {'learning': 1},

  'is': {'interesting': 1}
}
```

This table represents the Markov chain.

7. Calculate Transition Probabilities

```
probabilities = {}
```

The program converts frequencies into probabilities.

```
total = sum(next_words.values())
```

Formula:

and not on earlier states:

This is called the **Markov Property**. It means only the present word is used to predict the next word.

Difference: Bigram vs Markov Model

Feature	Bigram Model	Markov Model
Stores next-word frequency	✅	✅
Calculates probabilities	Usually not	✅
State transition representation	Limited	Explicit
Probabilistic prediction	Basic	Strong
Basis for Hidden Markov Models	❌	✅

In this code, the Markov Model is essentially a **probability-based bigram model** where each word is treated as a state and transitions are represented using probabilities.

Applications of Markov Models

- Text Prediction
- Autocomplete Systems
- Speech Recognition
- Machine Translation
- Weather Prediction
- Financial Forecasting
- Hidden Markov Models (HMMs)
- Natural Language Processing

Time Complexity

- Corpus Processing: **O(N)**
- Transition Counting: **O(N)**
- Probability Computation: **O(T)** (T = number of transitions)
- Prediction: **O(K)** (K = possible next words)

Overall complexity: **O(N)**.